CHAPITRE n° 3

Maîtriser Git

Ce que vous allez apprendre dans ce chapitre :

- Manipulation des repository
- Utilisation des commandes Git

CHAPITRE n° 3

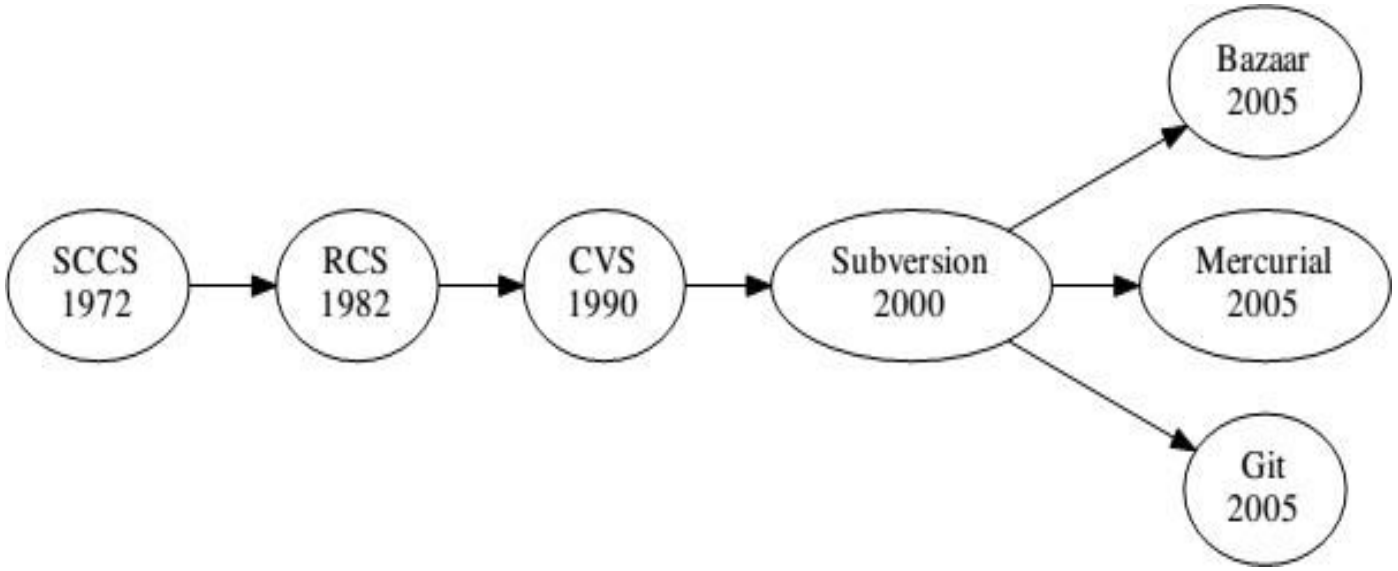
Maîtriser Git

1. **Introduction des différents outils de gestion de versioning**
2. Manipulation des repository
3. Utilisation des commandes Git

Introduction

Git :

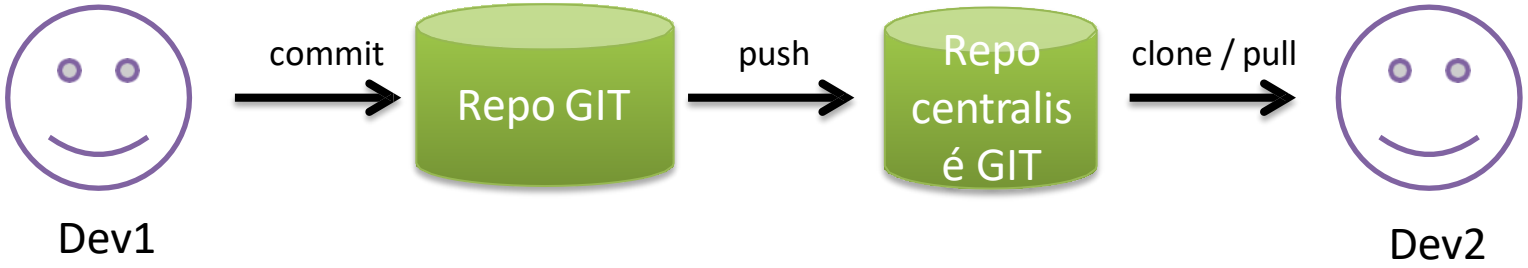
- Historique des modifications ;
- Résolution des conflits par la fusion :
- Possibilité d'avoir les branche.



Introduction

- Deux type de gestion de version:
  - Centralized Revision Control System (CVCS) - Système centralisé de contrôle des révisions ;
  - Distributed Version Control System (DVCS) - Système de contrôle de version distribué.

SVN : système de versioning centralisé (perte d'informations)



Git : système de versioning distribué (Historique à chaque utilisateur)

Centralized Revision Control System (CVCS)

- Architecture client serveur ;
- Un seul repository ;
- Un serveur central contient toutes les données ;
- Beaucoup de requêtes entre le client et le serveur (assez lent) ;
- Dépendance du serveur (check out) ;
- **Exemples** : CVS, SVN…

Distributed Version Control System (DVCS)

- Toutes les données sont sur notre machine ;
- Les opérations sont très rapides ;
- Connexion internet seulement pour les pull et push ;
- **Exemples** : Git, mercurial…

Contrôle de version

Un système de contrôle de versions vise à :

- Garder un historique des différentes versions des fichiers d'un projet ;
- Permettre le retour à une version antérieure quelconque ;
- Garder un historique des modifications avec leur nature, leur date, leur auteur ;
- Permettre un accès souple à ces fichiers, en local ou via un réseau ;
- Permettre à des utilisateurs distincts et souvent distants de travailler ensemble sur les mêmes fichiers.

CHAPITRE n° 3

Maîtriser Git

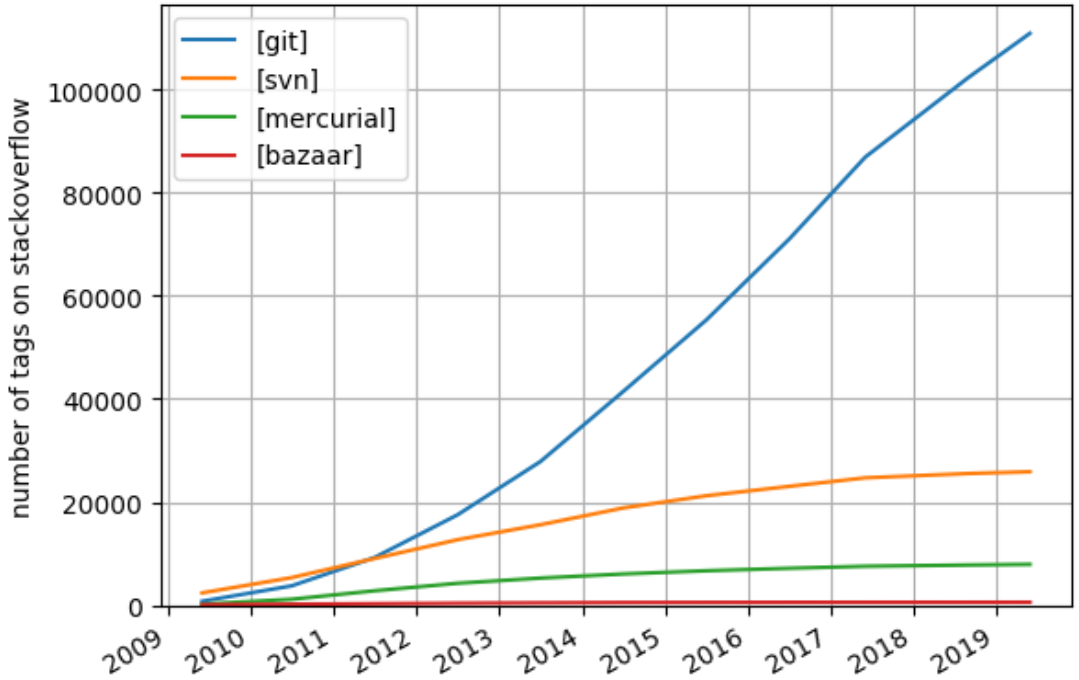
1. Introduction des différents outils de gestion de versioning
2. **Manipulation des repository**
3. Utilisation des commandes Git

Historique GIT

- Créé en avril 2005 par Linus Torvalds ;
- Objectif : gérer le workflow d'intégration des patches du noyau Linux ;
- Remplacer BitKeeper ;
- En Mai 2013, 36 % de professionnels utilise Git.

Historique GIT

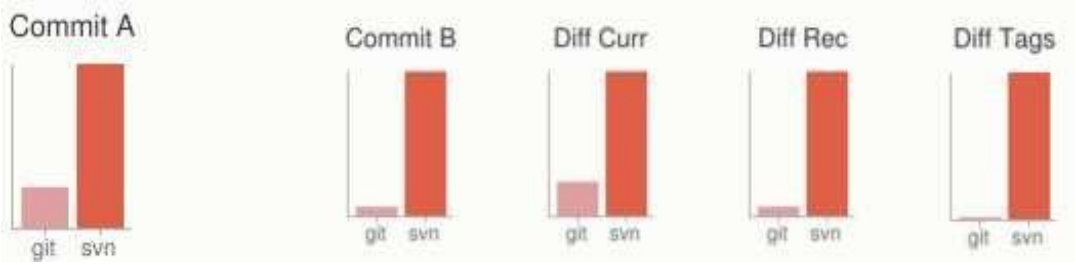
- Créé en avril 2005 par Linus Torvalds ;
- Objectif : gérer le workflow d'intégration des patches du noyau Linux ;
- Remplacer BitKeeper ;
- En Mai 2013, 36 % de professionnels utilise Git.



Pourquoi GIT ?

Pourquoi utiliser Git face à d'autres solutions ?

- Git a de nombreux avantages face à de plus anciens systèmes comme CVS et Subversion (SVN).
- Git est extrêmement simple d'utilisation, notamment couplé à une interface graphique comme GitHub.
  - Git est rapide :



  - Travailler hors-ligne : Git ne requiert pas de serveur distant pour fonctionner.
  - Et bien entendu, Git est gratuit et open source.

Les entreprises et les projets qui utilisent Git

Terminologie

- Le dépôt (repository) abrite tous les projets gérés sur un serveur. Il archive toutes les versions des fichiers stockés ;
- Au sein d'un dépôt se trouvent un ou plusieurs projets. Les projets contiennent des fichiers organisés en arborescence ;
- Les copies locales ou copies de travail correspondent aux espaces de travail (workspaces) des développeurs ;
- Chaque modification faite au dépôt constitue une révision ou version. Le numéro de révision commence à 1 et augmente de 1 à chaque opération.

CHAPITRE n° 3

Maîtriser Git

1. Introduction des différents outils de gestion de versioning
2. Manipulation des repository
3. **Utilisation des commandes Git**

Git configuration

- **Config :**
  - `git config --global user.name "Mohamed LACHGAR"`
  - `git config --global user.email "lachgar.m@gmail.com`

- **Initialisation :**
  - `mkdir formation`
  - `cd formation`
  - `git init`

- **Cloner le dépôt :**
  - `git clone "url du dépôt"`
  - `#On peut spécier un nom du dépôt`
  - `git clone url myrepository`

Git configuration (gitignore)

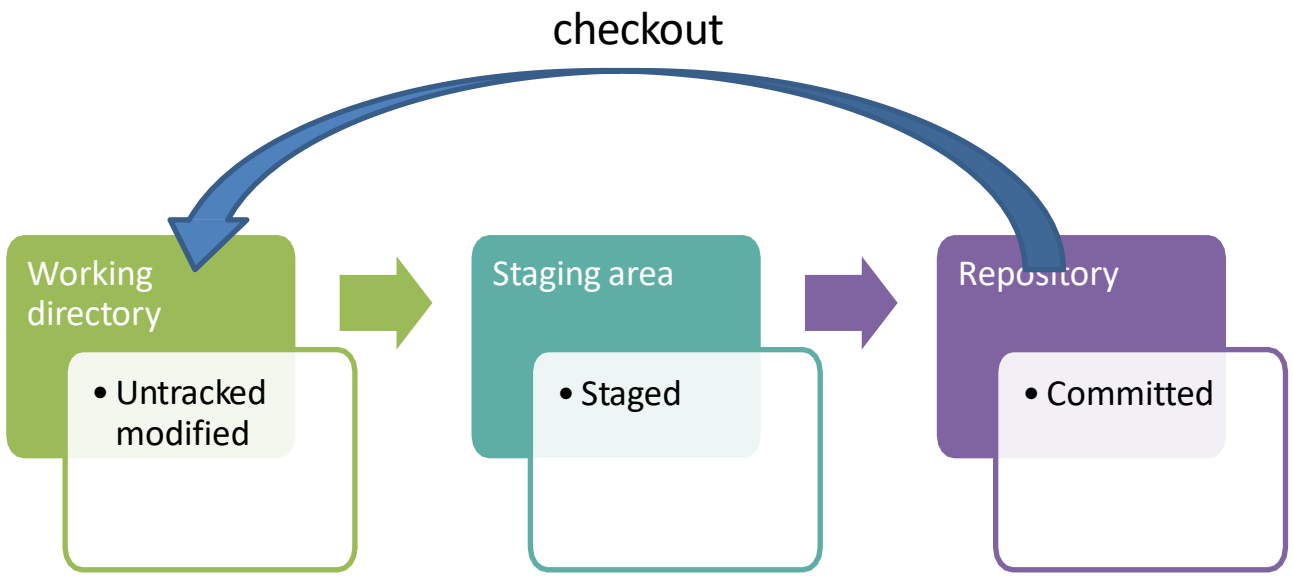
- gitignore :
  - données sensibles ou inutiles

- Créer/modifier le .gitignore à la racine du projet :
  - Exemples de fichiers à ignorer:
    - # /hybris/
    - /hybris/data
    - /hybris/log
    - /hybris/roles
    - /hybris/temp

- lib.so : le fichier lib.so sera ignoré ;
- *.class : tous les fichiers en .class seront ignorés ;
- conf/ : tous les fichiers du dossier conf seront ignorés ;
- conf/**/*. yml : tous les fichiers . yml de tous les sous-dossiers de conf/ seront ignorés.

Git workflow

- Status du fichier



Git workflow

- Staging area

```
#affiche le statut de la working directory et de la staging area
git status
# ajoute un fichier à la staging area
git add
# : unstage un nouveau fichier
git rm --cached
git checkout --: retire un fichier de la staging area
```

Git workflow

- **Commit**
  - Un commit est pointeur sur branch ;
  - Chaque commit est unique :

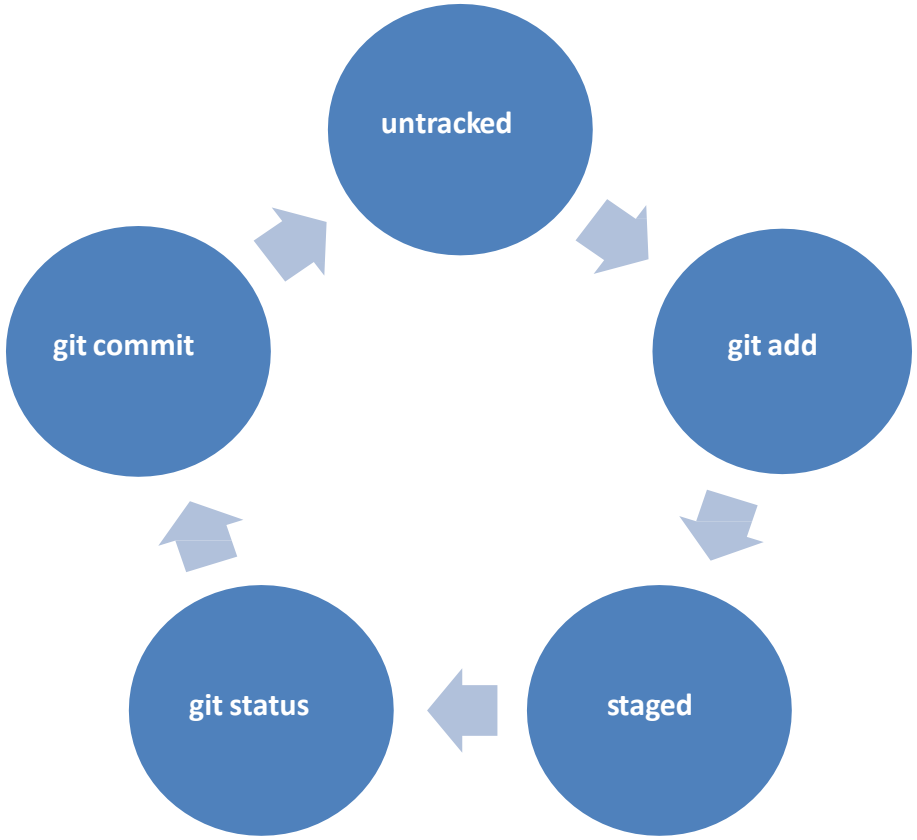
```
git commit -m "mon commentaire de commit"
# génère un commit avec les modifications contenues dans
#la staging area

git commit -a -m "mon commentaire de commit"
# ajoute tous les fichiers modifiés (pas les ajouts /
#suppressions) à la staging area et commite)
```

- **Revert**

```
# Revert a pushed commit
git revert <commit_hash>
```

Git workflow

- Status fichier résumé



Routines Git

- **Git status**

La **commande git status** affiche la liste des fichiers modifiés ainsi que les fichiers qui doivent encore être ajoutés ou validés.

Usage :

git status

- **Git add**

La **commande git add** peut être utilisée pour ajouter des fichiers à l'index. Par exemple, la commande suivante ajoutera un fichier nommé temp.txt dans le répertoire local de l'index :

git add temp.txt

Routines Git

- **Historique: git log**

L’ **exécution de** la **commande git log** génère le log d’une branche.

Un exemple de sortie :

commit 15f4b6c44b3c8344caasdac9e4be13246e21sadw Author: Mohamed LACHGAR <lachgar.m@gmail.com>

Date: Mon Oct 1 12:56:29 2019 -060

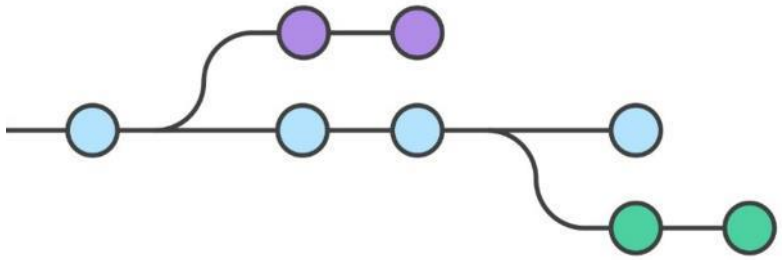
- **Git push**

Git push est une autre commandes GIT de base. Un simple push envoie les modifications locales apportées à la branche principale associée :

git push origin master #branche master

git push origin nouvelleBranche # nouvelle branche

Gestion des branches



```
git push origin master #branche master
git push origin nouvelleBranche # nouvelle branche
```

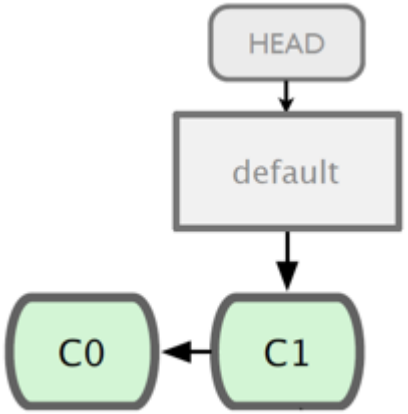
- **Mise à jour du branch**

```
# J'ai developpé et commité sur ma branche locale
# Mais l'origine de ma branche (le master
# par exemple) a  également évolué !

git checkout master
git pull
git checkout nouvelleBranche
git rebase master nouvelleBranche
```

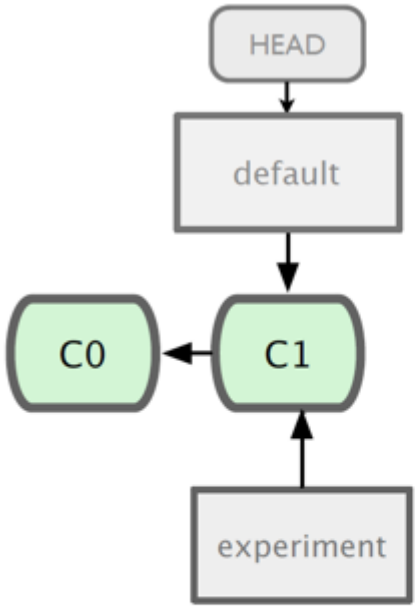
Gestion des branches

```
git checkout master
# Je dev sur le master (alors que j'aurai du créer une branche)
…
git commit -am "Je commit sur le master"
# Je m'en aperçoit
git branch newFeature
# Maintenant je dois remettre le master « clean »
# Si je veux quand même conserver les fichiers
git reset --soft HEAD~1

# Si je ne veux pas conserver les fichiers
git reset --hard HEAD~1
```

Gestion des branches

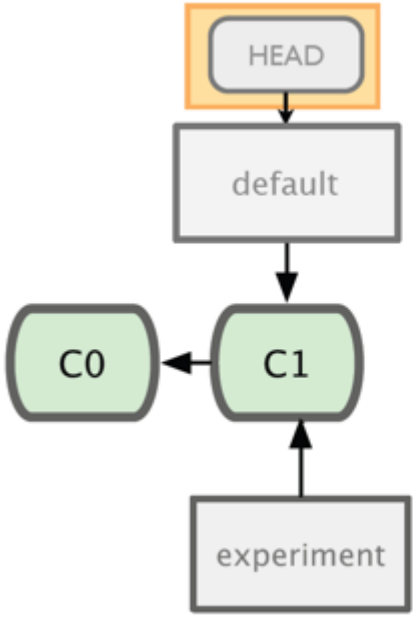


Gestion des branches



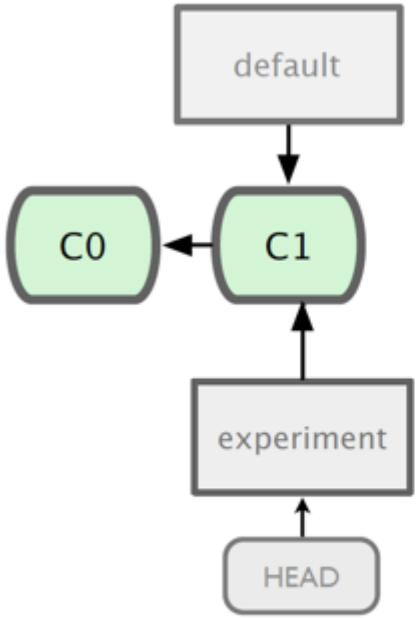

```
git branch experiment
```

Gestion des branches



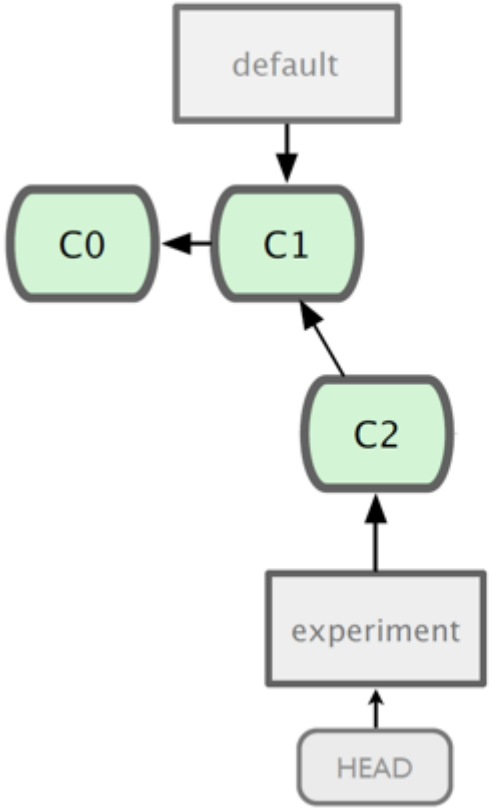

```
$ git branch
* default
  experiment
```

Gestion des branches



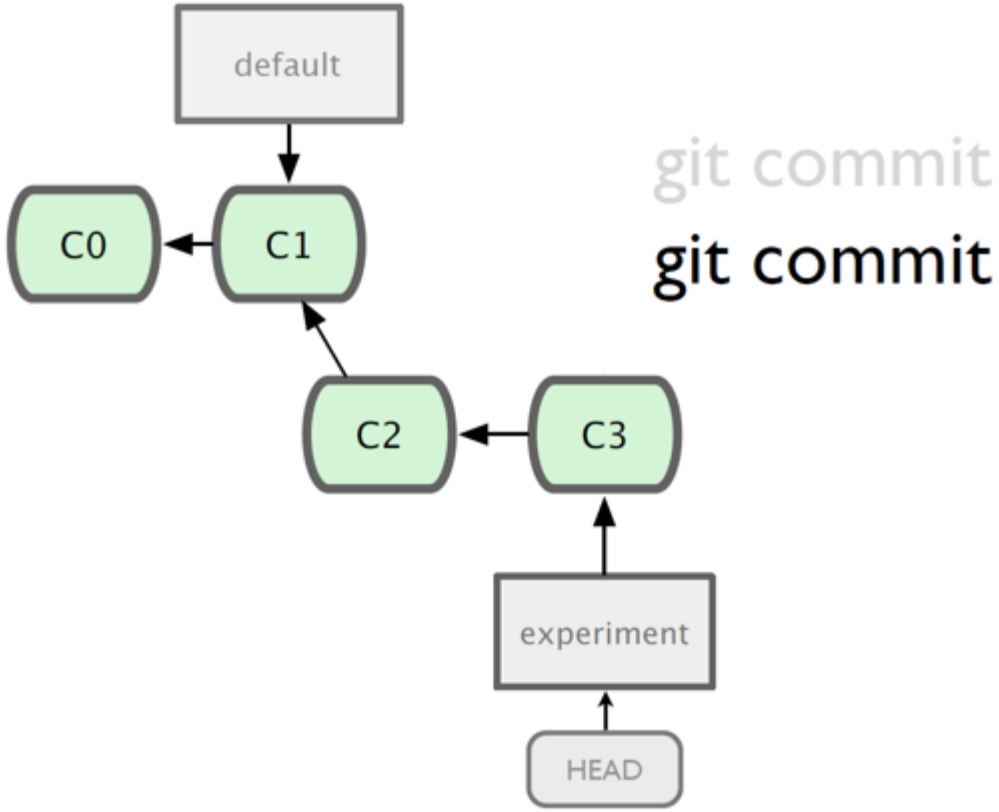
git checkout experiment

Gestion des branches

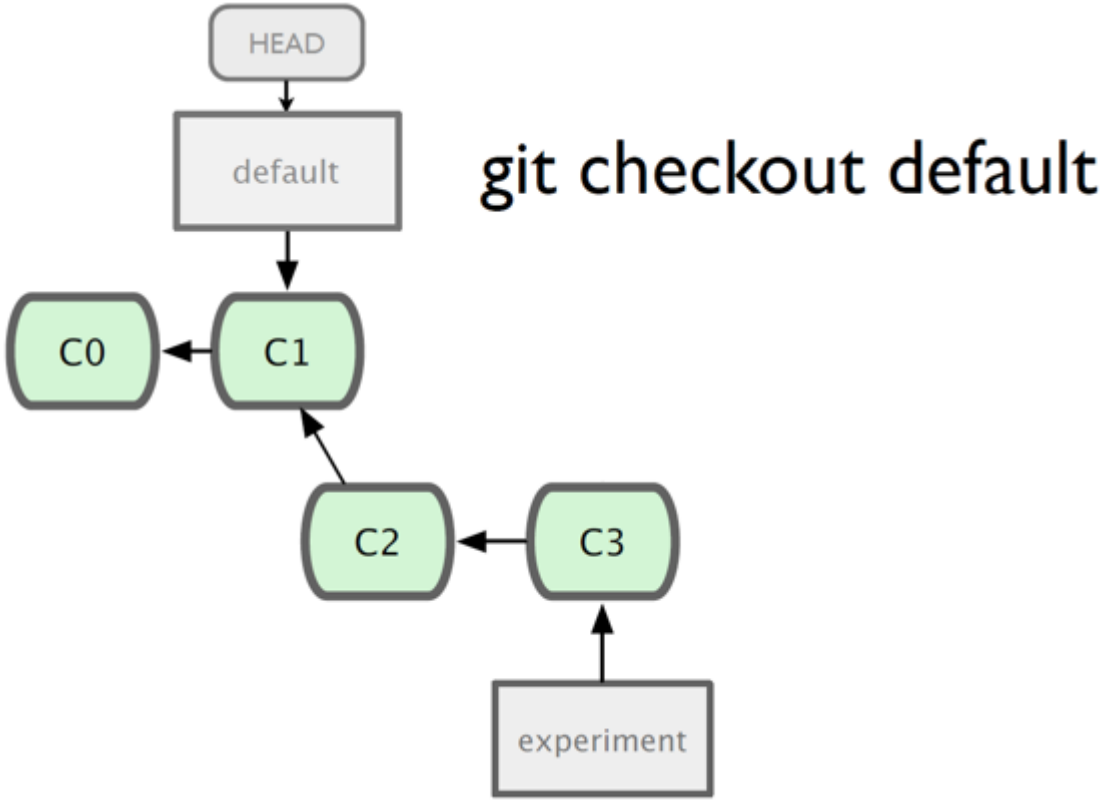


git commit

Gestion des branches

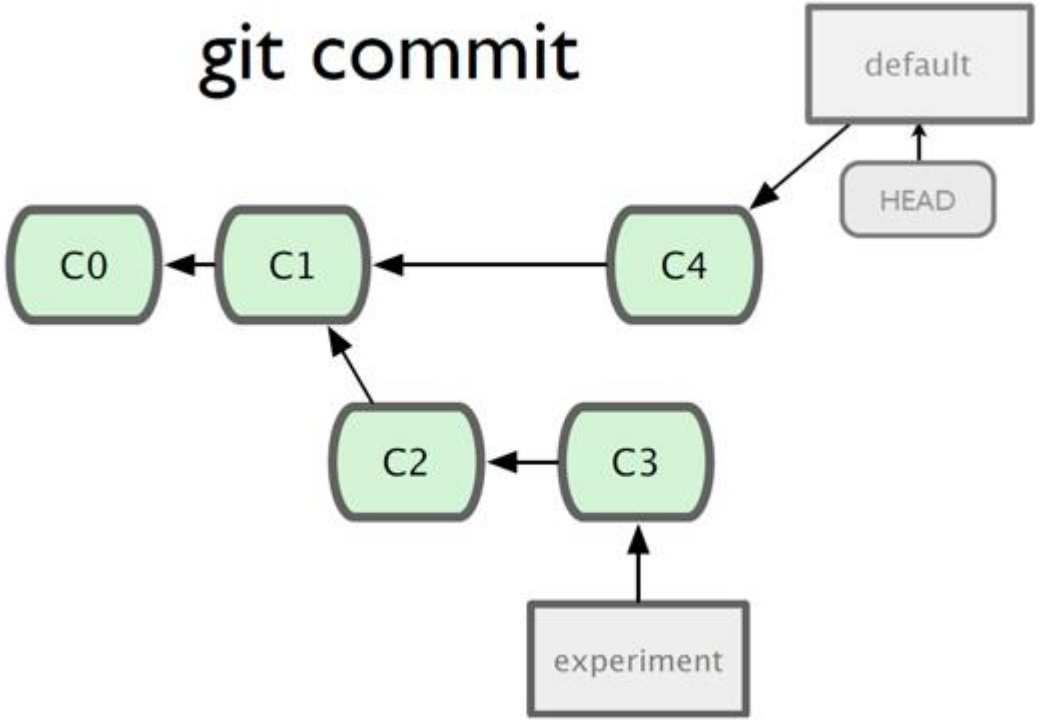


Gestion des branches

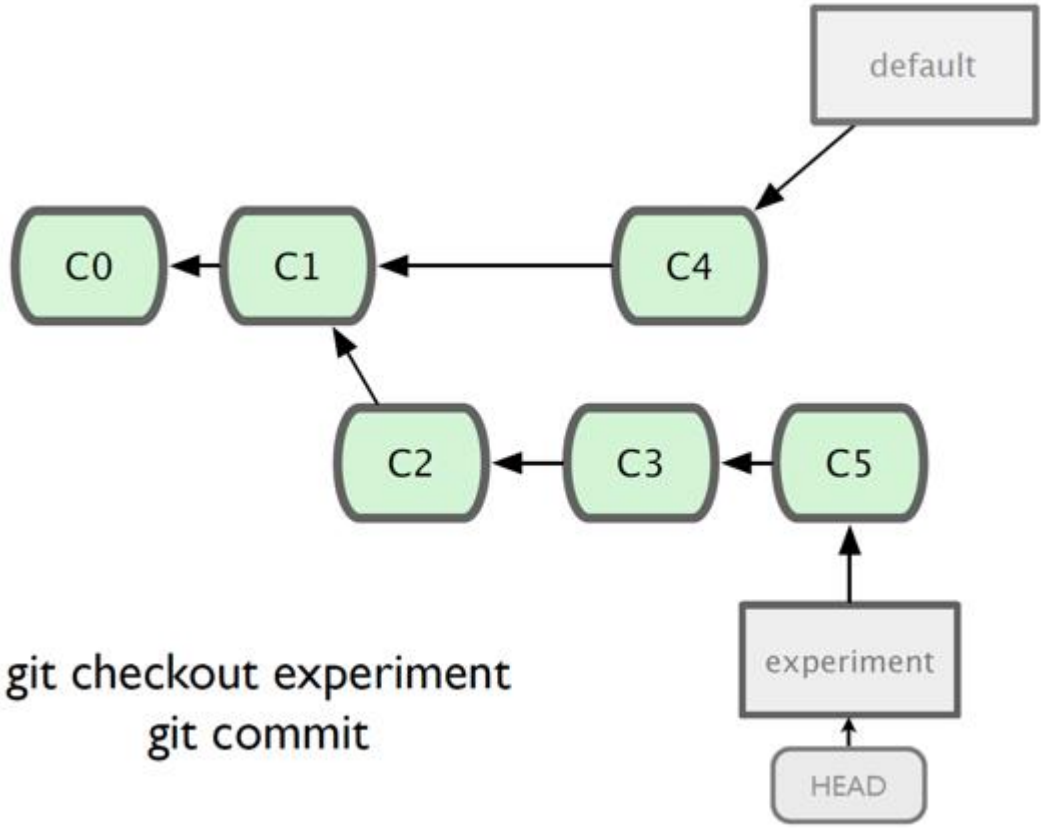


git checkout default

Gestion des branches



Gestion des branches



git checkout experiment
git commit

Gestion des branches

- **Lister les branches:**
  - git branch
- **Récupérer les branches distantes:**
  - git fetch
    - ➢ Permet à un utilisateur d'extraire tous les fichiers du dépôt distant qui ne sont pas actuellement dans le répertoire de travail local ;
    - ➢ Exemple d'utilisation: **git fetch origin**
- **Lister les branches distantes:**
  - git branch –a
- **Supprimer une branche**
  - git branch –d <nom-branche>

Gestion des branches

- **Gestion des merges :**
  - git merge branch1

  La **commande git merge** est utilisée pour fusionner une branche dans la branche active.

- **Gestion des conflits**
  - Git crée un conflit lorsqu'il ne sait pas quelle version d'un fichier il doit conserver. Cela arrive quand un fichier a été modifié au même endroit par 2 sources distinctes !
  - **git status** permet de comprendre la nature du conflit : c'est votre point de départ pour le résoudre.
  - À vous de choisir la solution qui vous convient le mieux pour résoudre votre conflit :
    - conserver la version distante ;
    - conserver la version locale ;
    - faire une version adaptée à la circonstance.